Google Cloud H2S

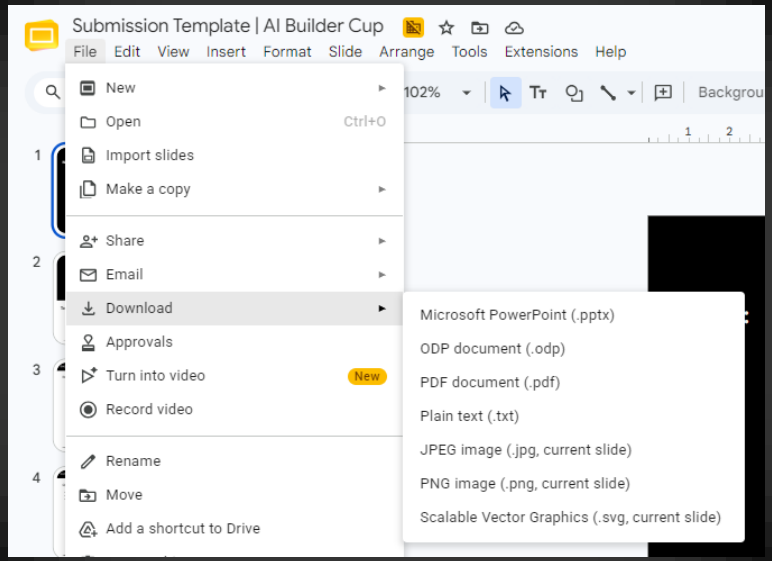
AI Builder Cup

Build in JAPAC.
Build for the world.



Team Details

Team name: EmbedDesk
Team leader name: Ryan Wan
Problem Statement: Small and mid-sized businesses and device vendors cannot easily embed enterprise-grade, multi-channel AI customer support into their websites. EmbedDesk turns any site into an intelligent, ticket-generating support surface with one line of code.



Note: Please download this template to make your edits and submit the final version.

To download, go to File > Download > Microsoft PowerPoint (.PPTX). Kindly refrain from sending edit requests.

Brief about the idea

EmbedDesk — an embeddable AI customer-service agent built on Google Cloud.

A single <script> tag drops a conversational support widget into any website. The agent answers from a Retrie

One shared LLM core serves every channel — web, embedded iframe, email (IMAP+SMTP) and instant messa

Everything runs serverlessly on Google Cloud — Cloud Run or Firebase Cloud Functions (the deployed prototy

Opportunities

- How different is it from any of the other existing ideas?
- How will it be able to solve the problem?
- USP of the proposed solution

How different? Developer-first and multi-channel: a real tool-use agent (search knowledge, query order, get customer, create ticket) with a deterministic human-escalation path — deployable as code, not a walled-garden chatbot builder.

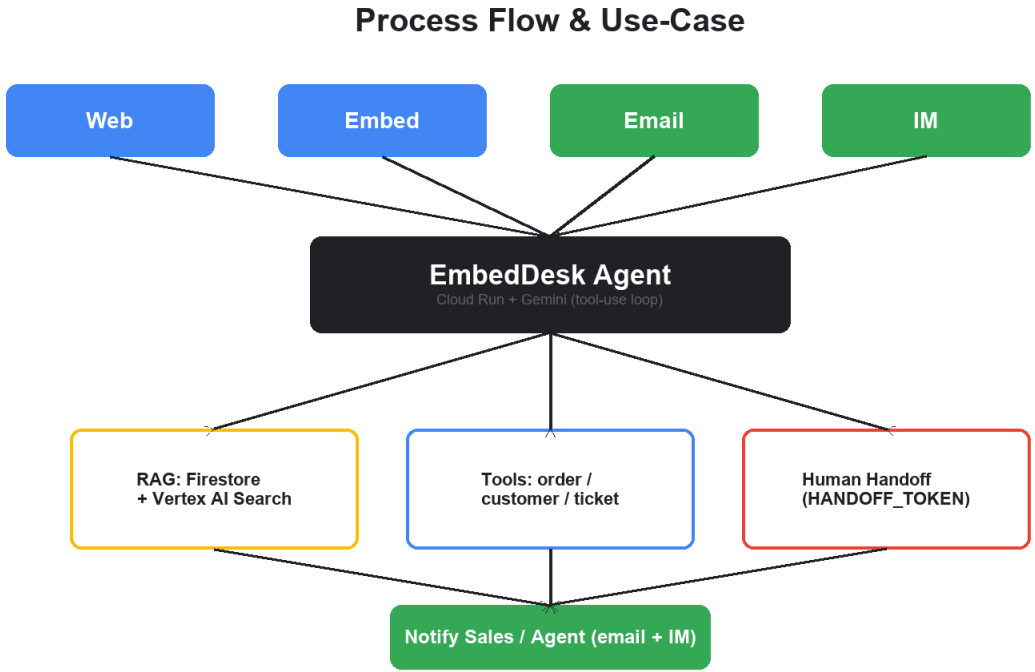
How it solves the problem: one embed snippet replaces a whole help-desk stack — RAG answers, order/customer lookups, auto-created tickets with sentiment tagging and SLA, and sales notification, all on serverless Cloud Run.

USP: "One line of code, expert-grade support on any site." Deterministic guardrails (knowledge-gap detection, escalation workflow, auto priority/SLA) make it safe for real customer-facing use — not just a demo.

List of features offered by the solution

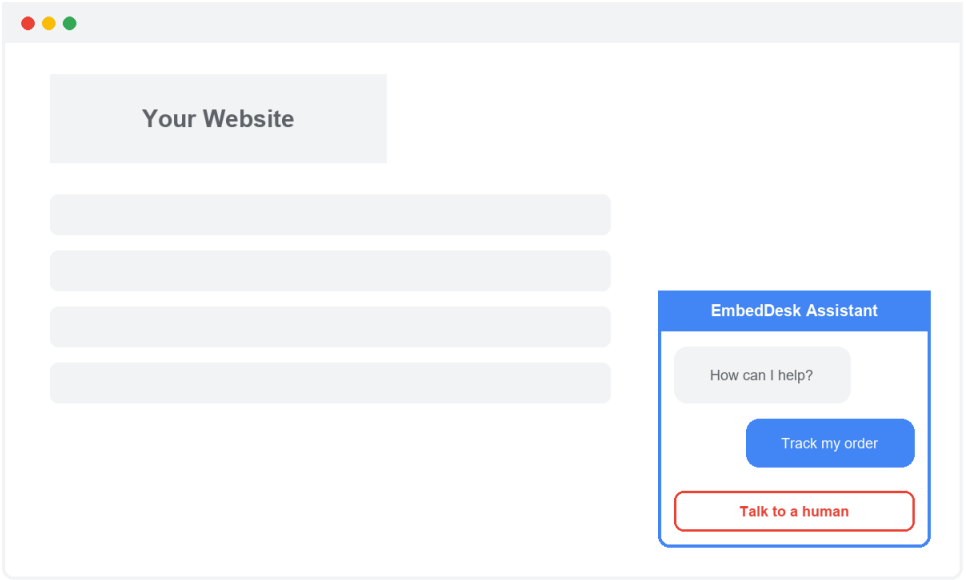
1. One-line embeddable chat widget (Shadow DOM, themeable, works on any site)
2. RAG knowledge base — synonym / colloquial query expansion + optional semantic re-ranking
3. Tool-using agent: search_knowledge · query_order · get_customer · create_ticket
4. Multi-channel: web · embed · email (IMAP+SMTP) · IM — one shared agent core
5. Deterministic human handoff (HANDOFF_TOKEN) + agent-created tickets with dedup guard
6. Sentiment detection + smart escalation (automatic priority and SLA)
7. Agent Copilot — real-time reply suggestions for human agents
8. CSAT surveys, insights dashboard (BigQuery) and full LLM call observability

Process flow diagram or Use-case diagram

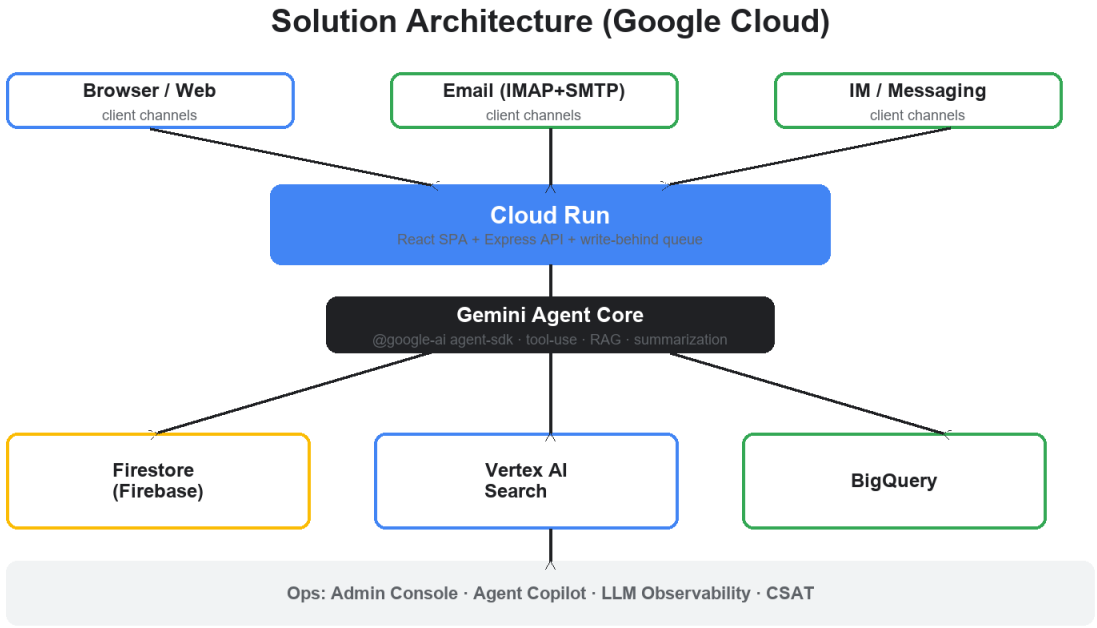


Wireframes/Mock diagrams of the proposed solution (optional)

Embeddable Widget Wireframe



Architecture diagram of the proposed solution



Technologies to be used in the solution

Technologies actually used in the submitted solution:

Google Cloud serverless — Cloud Run or Firebase Cloud Functions host the web frontend + API backend; the deploye

Google Gemini — the large language model powering the agent: tool-use loop, RAG answering, summarization.

Firestore (Firebase) — serverless NoSQL datastore for the knowledge base, sessions, tickets and RAG vectors — the

Firebase — the app data platform wrapping Firestore; Firebase Auth / Hosting are the planned next-step integrations.

Vertex AI Search — semantic retrieval (RAG) over the knowledge base with re-ranking.

BigQuery — analytics and insights over ticket and conversation data.

All cloud services are environment-gated: the deployed demo instance runs the same code on local SQLite + lexical R

Estimated implementation cost (optional)

Estimated implementation cost: ~$0 per month at demo scale.

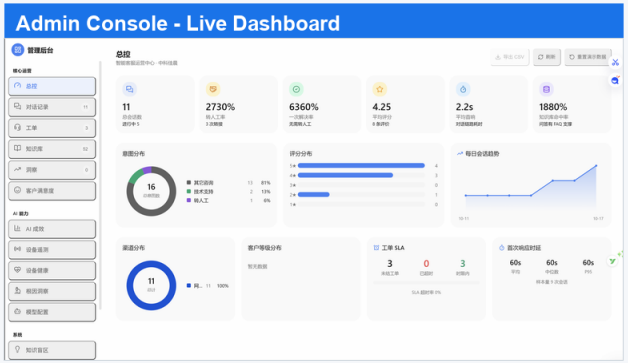
Firebase Blaze free tier (2,000,000 function invocations / month) covers the prototype; scale-to-zero when idle.

Gemini API free tier / pay-per-token; Firestore and BigQuery stay within their free allowances.

No minimum spend. Deleting the Firebase project after evaluation stops all cost immediately.

Snapshots of the prototype

Admin Console - Live Dashboard



Agent Workbench - Reasoning Trace



Auto-created Tickets + SLA



RAG Knowledge Base (52 FAQs)



Prototype Performance report/Benchmarking

Load test (local, SQLite mode):

~492 requests/s sustained ingest at 100 concurrent connections — p95 latency 267 ms, 0 errors.

~360 requests/s at 400 concurrent connections — no avalanche, graceful degradation.

Bottleneck fixes applied: knowledge-base pre-computation + LRU cache (5,000 entries) + write-behind batch queue.

Observability: every LLM call logged with provider / model / latency / token usage / status (ok · degraded · error).

RAG answer hit-rate and KB cache hit-rate tracked as live metrics.

Additional Details/Future Development (if any)

Current status: production-ready codebase — 9 automated verification suites, all green.

Roadmap:

Firebase Auth for end-customer identity and Firestore multi-tenant data isolation.

Firebase Hosting + Firebase Extensions for one-click install from the Firebase console.

WebSocket token streaming for real-time responses; multilingual support.

Mobile SDK and a self-serve onboarding wizard for non-developers.

Provide links to your:
GitHub Public Repository: https://github.com/Aqr325/embeddesk (branch: hack2skill)
Demo Video Link (3 Minutes): https://youtu.be/N7_jOGu725s — embeddesk-demo.mp4 (3 min 29 s, 1920×1080)
Final Product Link: https://embeddesk.web.app (Firebase Hosting + Render API)

Google Cloud H2S

AI Builder Cup

Build in JAPAC.
Build for the world.

Thank you

